LRN Payer Policy Intelligence & Claim Policy Chatbot

Technical Design & Build Decision Document · Version 4.0 · August 2026

Final edition — architecture design (v2.0) + payer policy source review (v3.0) + technology decisions, costing and development breakdown

Application: LRN Web Application (ASP.NET MVC, Razor, Bootstrap 5) | **Background:** Python workers | **Data:** SQL Server / Azure SQL | **AI:** Azure AI Document Intelligence, Azure OpenAI / Foundry, Azure AI Search
Evidence base: 14 sample payer policies, 7 payer entities, 852 pages | **Status:** For approval — decisions recorded in section 21

0. Version history and what v4.0 adds

> **v4.0 adds the build decisions.** Sections 5–8 add the numbered requirement breakdown, the PDF-extraction technology decision (**Python, not .NET** — section 6), the full stack decision table and the deployment architecture. Sections 16–18 add the performance design, the cost model with worked figures, and the development breakdown by work package and effort. Everything else carries forward from v3.0 unchanged. The table below records what v3.0 changed against v2.0, and remains the reference for why the data model looks the way it does.

Version 2.0 was written from the architecture side: layer boundaries, ownership, security, and how the chatbot fits into LRN. The source review was written from the document side: fourteen real payer policies were read and the structural consequences recorded. The two agree on approach and disagree on the **shape of the data** — and the source review is right, because it was derived from actual documents rather than from an assumed model.

Area	v2.0 said	Source review found	v3.0 decision
Grain of a coverage row	One rule per CPT code under a policy version	The same code appears legitimately more than once at different statuses — BCBS Michigan 87797 is established for some organisms and investigational elsewhere	**Adopt the review's grain.** A row is `version × indication_group × code × analyte × method × product_line`. Deduplicating on code alone silently destroys correct data.
ICD storage	ICD codes attached to each rule	Aetna CPB 0650 carries ~633 ICD codes across ~15 indication groups; per-code storage explodes to six figures of meaningless rows	**Adopt named ICD sets** attached to the indication group, with polarity (covered / not covered) and range members.
Coverage status	Four values: Covered / NotCovered / Conditional / Unknown	13 distinct status labels across 7 payers; also a genuine difference between "not covered" and "policy is silent"	**Expanded canonical enum** plus a per-payer `status_vocabulary` synonym table. `NO_STATEMENT` becomes a first-class value.
Payer as the key	PayerId resolves the policy	BCBS Arkansas states two different positions for the same test depending on whether the member's contract has primary coverage criteria; the policy names eight plan families	**Add a product-line / contract-tier dimension** (nullable) and a benefit-manager dimension.
Payer format handling	"The canonical model absorbs payer differences"	Fourteen documents collapse into **four structural archetypes**; extraction should key off the archetype, not the payer name	**Adopt archetype detection** with archetype-specific prompts. A new payer writing a determination matrix works on day one.
What the LLM reads	The whole normalised document	60–80% of each document is evidence review, background and references — no coverage facts, but it dominates token cost and dilutes attention	**Deterministic section segmentation first.** Only criteria and coding sections reach the model, one indication group per call.
Retired policy text	Not addressed	BCBS Arkansas 2010035 contains superseded blocks dated 2011 and 2022 inside the current PDF	**New: effective-date scoping pass** before extraction. Only the live block is extracted; the rest is archived as history.
Criteria logic	Free-text `MedicalNecessity` field	Anthem and Centene write explicitly nested AND/OR trees — the only machine-readable logic in the corpus	**Preserve the tree** in a `criteria_node` table. Flattening it to a text cell destroys the one structured thing the payer gave us.
Payment rules	Treated as coverage conditions	BCBS Illinois and Ambetter documents are claim-level payment logic — quantity limits and mutual exclusion, not coverage status	**Separate** `reimbursement_rule` table. These evaluate against claim lines, not against a coverage question.
Confidence	Composite trust score	Every fact must carry a page number and a verbatim quote; a validator checks the quote appears in the source	**Both.** Verbatim grounding is a hard gate — unquotable rows are discarded, not scored. Trust score orders the review queue.
Review UI	Razor screen inside LRN	Streamlit or small FastAPI service — "not the part that deserves a bespoke front end"	**Keep it in LRN.** The reviewers are LRN users, the role framework and audit already exist, and the crosswalk work belongs beside the panel data. Revisit only if Phase 3 slips.
Claim integration	Core feature — "policy for claim 123456"	Not covered; the review stops at code-level lookup	**Retained from v2.0** and extended: see section 13, where the new grain changes how a claim resolves to a coverage line.
Existing master file	Untouched; AI proposes changes only in a future phase	Adds an Excel migration phase with a variance report between the workbook and the source documents	**Both, in order:** reconcile first (Phase 4), propose changes later (Phase 7).

Change detection	New PDF arrives by email; hash comparison	Scheduled metadata check against each policy URL; download only on real change	**Both paths** — push (email) and pull (URL polling).

Carried forward unchanged from v2.0: the AI / Python / .NET / Human ownership split, PHI handling, Payer Matching Engine reuse, lab scoping, the LRN panel/test crosswalk, chat audit and token budgets, and the rule that the existing Python validation worker is not modified.

1. Scope

In scope

- **A queryable coverage database.** Payer policy documents are ingested, segmented, extracted into a canonical coverage model by an LLM, validated, approved by a human, and published as versioned, effective-dated SQL data.
- **A policy chatbot inside LRN** answering three kinds of question — claim, lookup and narrative — every answer citing payer, policy, version, effective date and page.

Out of scope for v1

- No change to the existing Python payer-policy validation worker or the master file it consumes.
- No claim adjudication, no automatic appeal, no automatic update of production master data.
- No text-to-SQL. Lookup runs through parameterised intents over read-only views, never generated SQL.

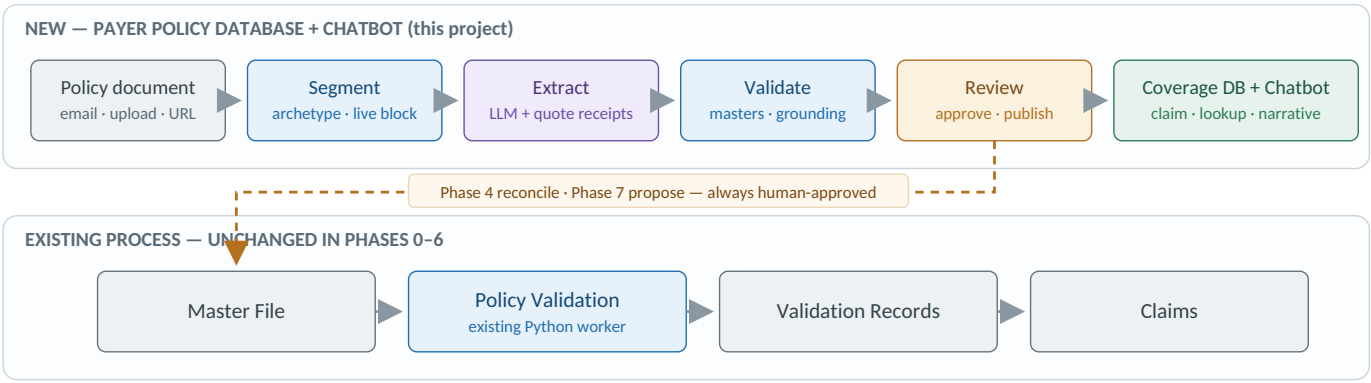

Figure 1 — The new database is additive. The deterministic validation chain is untouched.

2. Evidence base — what the sample documents actually look like

Fourteen policies from seven payer entities were read end to end. All are born-digital and extracted cleanly to text, so OCR was not needed for this set — but the sample is deliberately varied, spanning a four-page payment policy and a 348-page bulletin. Every design decision in this document traces to something observed in these files.

14	852	7	4	13	348
DOCUMENTS	PAGES	PAYER ENTITIES	ARCHETYPES	STATUS LABELS	PAGES, LARGEST POLICY

Payer	Policy #	Subject	Category	Currency
Aetna	CPB 0215	Lyme disease & other tick-borne diseases	Clinical policy bulletin	Reviewed 04/09/2025
Aetna	CPB 0443	Cervical cancer screening & diagnosis	Clinical policy bulletin	Retrieved 10/13/2025
Aetna	CPB 0471	Tuberculosis testing	Clinical policy bulletin	Retrieved 11/05/2025
Aetna	CPB 0650	PCR testing: selected indications — **348 pages, ~173 CPT, ~633 ICD**	Clinical policy bulletin	Retrieved 01/12/2026
Aetna	CPB 1008	Infectious diseases: selected tests	Clinical policy bulletin	Retrieved 01/12/2026
Ambetter / Centene	CC.PP.009	Unlisted procedure codes	Payment policy	Reviewed 11/21/2024
Ambetter / Centene	CG.CP.MP.02	Concert infectious disease: multisystem testing	Clinical policy (lab vendor)	Rev. 11/2024 · v2025.1
Anthem	CG-LAB-10	Zika virus testing	Clinical UM guideline	Publish 10/01/2025
Anthem	CG-LAB-22	NAAT with algorithmic analysis — vaginitis	Clinical UM guideline	Publish 07/01/2025
BCBS Arkansas	2010035	Lyme disease IV antibiotic therapy & diagnostic testing	Coverage policy — pharmacy	Eff. 01/10/2024
BCBS Arkansas	2024058	RTM / onychomycosis testing	Coverage policy — laboratory	Eff. 02/01/2025
BCBS Illinois	CPCPLAB033	Diagnostic testing of influenza	Clinical payment & coding policy	Plan eff. 01/01/2026
BCBS Kansas	—	Identification of microorganisms using nucleic acid testing	Medical policy	Rev. 08/19/2025
BCBS Michigan	2196460	Identification of microorganisms using nucleic acid probes	Joint medical policy (BCBSM/BCN)	Eff. 11/01/2025

Three observations shape the build. **Aetna CPB 0650 alone is 41% of the sample** and carries roughly 633 distinct ICD-10 codes across thirteen indication groups — which is exactly why the current spreadsheet keeps ICD at sheet level rather than row level. **BCBS Kansas and BCBS Michigan** cover the same clinical territory with near-identical code sets but entirely different status vocabularies, making them the ideal pair for validating the canonical status mapping. **The two Lyme policies** (Aetna 0215 and BCBS Arkansas 2010035) are the natural cross-payer comparison the chatbot will be asked for on day one.

3. Four document archetypes

Fourteen documents collapse into four structural patterns. This matters more than the payer name: **the extraction prompt keys off the archetype, not off "is this Aetna".** A new payer that writes a determination matrix gets the matrix prompt on day one, with no new parser.

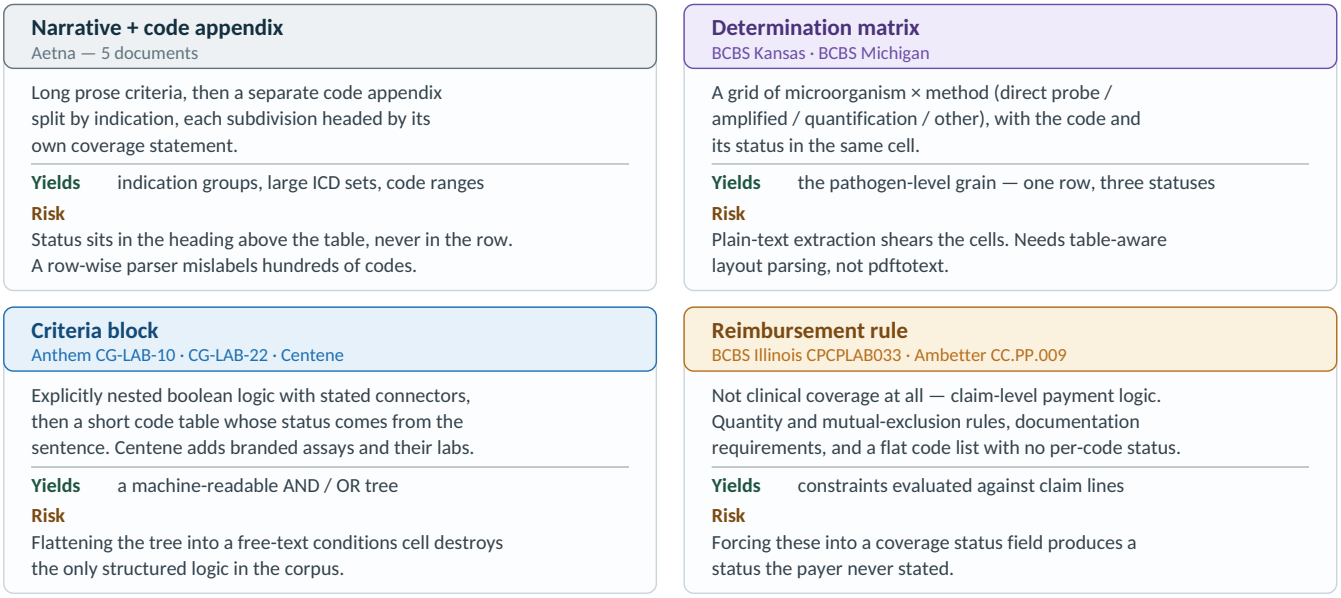

Figure 2 — The four archetypes. Each gets its own prompt and its own target tables.

4. Where extraction breaks — observed, not hypothetical

Each item below was seen in the sample set. Each is a way that “send the PDF to an LLM and ask for a table” produces confidently wrong coverage data — which in this setting means a wrong billing decision.

#	Observed problem	Design fix
1	**Thirteen words for the same thing.** Across seven payers: medically necessary, established (EST), med nec, meets primary coverage criteria, experimental/investigational/unproven, investigational (INV), E/I, not medically necessary, does not meet primary coverage criteria, not reimbursable, covered if selection criteria are met, and more.	Canonical status enum plus a per-payer synonym table seeded from these fourteen documents. An unmapped label is an exception, never a guess at the enum. The raw label is stored alongside so nothing is lost.
2	**One test, two statuses, by contract.** BCBS Arkansas 2024058 states NAAT for onychomycosis does not meet member benefit criteria, and separately that for members whose contracts lack primary coverage criteria it is considered differently. The policy enumerates eight plan families.	Coverage cannot be keyed on payer alone. Add a nullable `product_line` dimension with a `has_primary_coverage_criteria` flag. Null means “applies to all products”.
3	**Retired text inside the current PDF.** BCBS Arkansas 2010035 contains a current effective block plus superseded windows and clauses stamped 2011 and 2022.	A date-scoping pass before extraction: detect effective-date banners, keep only the block whose window covers today, archive the rest as history.
4	**The same code twice with different statuses — correctly.** BCBS Michigan lists 87797 as established for covered organisms with a unit qualifier, while 87799 sits under investigational; BCBS Kansas splits status across organisms.	The fact grain is code × analyte × method × product line. Deduplicating on code alone silently deletes true rows.
5	**Ranges, add-on markers and bracketed qualifiers.** Aetna writes `81206 - 81208` and `A69.20 - A69.29`, marks add-on codes with a leading `+`, and hides restrictions in square brackets inside descriptions.	Store the range as written and expand against the code master at load time. Keep the bracketed qualifier in its own field, not glued into the description string.
6	**The pathogen dimension is not in the code list.** In BCBS Kansas the organism exists only in the matrix; the code appendix at the back is organism-free. In Aetna the indication exists only in the sub-heading. In Centene the branded test exists only in a reference table.	Extraction is a join, not a scrape. Parse matrix, criteria blocks and code appendix separately, then merge deterministically into coverage lines.
7	**60–80% of every document is evidence review.** In Aetna CPB 0650 the criteria and coding sections are roughly 17% of the extracted text. The rest is background, rationale, discussion and references — no coverage facts, but it would dominate token cost and dilute attention.	Deterministic section segmentation before the model sees anything. Send only policy/criteria and coding sections. A shorter, denser prompt also extracts more accurately.
8	**Line-wrapped descriptions and column-aligned tables.** Plain text extraction splits one code description across several lines and renders matrices as space-aligned columns; naive line-by-line parsing truncates descriptions and shears cells.	Layout-preserving extraction with table geometry (PyMuPDF + pdfplumber), escalating to Azure Document Intelligence for matrix pages and any scanned document.
9	**Payer names on documents do not match the payer master.** (Carried from v2.0.)	Resolve through the existing Payer Matching Engine. Unresolved payers go to the exception queue, never guessed by the LLM.

5. Requirement breakdown

Requirements are grouped by module. **Priority** is M for the MVP set that must exist before the chatbot is credible, and P2 for the following release. Every requirement maps to a work package in section 18.

5.1 Functional requirements

ID	Requirement	Pri	Phase
M1 — Ingestion and registration			
FR-1.1	Accept policy documents by manual upload in LRN (PDF, DOCX, XLSX, HTML)	M	1
FR-1.2	Monitor a shared payer mailbox and ingest qualifying attachments	P2	6
FR-1.3	Poll registered payer policy URLs on a schedule and ingest on change	P2	6
FR-1.4	Detect duplicates on three keys: file SHA-256, normalised-text hash, and `PayerId + PolicyNumber + EffectiveDate`	M	1
FR-1.5	Resolve the payer through the existing Payer Matching Engine; an unresolved payer raises an exception and never proceeds on a guess	M	1
FR-1.6	Store the original document immutably in Blob with retrieval date, source and supplier metadata	M	1
FR-1.7	Register a document row and enqueue for processing; ingestion is never synchronous	M	1
M2 — Document understanding			
FR-2.1	Extract text with page and word coordinates preserved	M	1
FR-2.2	Extract table geometry, including tables that span pages and rotated tables	M	1
FR-2.3	Escalate matrix pages and any scanned page to Azure Document Intelligence Layout	M	1
FR-2.4	Classify sections into criteria, coding and background; only criteria and coding proceed to extraction	M	1
FR-2.5	Detect the document archetype (narrative+appendix, determination matrix, criteria block, reimbursement rule)	M	1
FR-2.6	Effective-date scoping: identify the live block, archive superseded blocks as history, never extract retired text	M	1
FR-2.7	Persist normalised markdown and per-page JSON so later stages can re-run without re-reading the PDF	M	1
M3 — Extraction and validation			
FR-3.1	One model call per indication group, with the archetype prompt and that payer's status lexicon supplied	M	2
FR-3.2	Force output through a JSON schema (structured output / tool use), not prose	M	2
FR-3.3	Every emitted fact carries `source_page` and a verbatim `source_quote`	M	2
FR-3.4	Validate code format and existence against CPT / HCPCS / PLA and ICD-10 masters	M	2
FR-3.5	Store code ranges as written and expand against the master at load time	M	2
FR-3.6	Map the raw status label to the canonical enum via the payer synonym table; an unmapped label is a reviewer exception, never a guess	M	2
FR-3.7	Discard any row whose quote does not appear verbatim in the source text	M	2
FR-3.8	Deterministically merge matrix, criteria block and code appendix into coverage lines	M	2
FR-3.9	Compute a trust score per row to order the review queue	M	2
FR-3.10	Retry once with the validator error appended before routing to the exception queue	M	2
M4 — Review and publication			
FR-4.1	Review queue scoped to one policy version, flagged rows first	M	3
FR-4.2	Side-by-side display: extracted row left, rendered source page right, quoted sentence highlighted	M	3
FR-4.3	Approve, edit, reject or send back for re-extraction	M	3
FR-4.4	Capture edits as structured corrections, not free text	M	3
FR-4.5	Bulk approve rows above the confidence threshold (disabled until Phase 6 evidence supports it)	P2	6
FR-4.6	Publish transactionally per version — wholly published or wholly held, never half-updated	M	3
FR-4.7	Close the prior version by effective date rather than deleting it	M	3
FR-4.8	Admin screens for the status vocabulary and the LRN panel/test-to-analyte crosswalk	M	3
FR-4.9	Record every extraction run and review action for audit	M	3
M5 — Query and chatbot			
FR-5.1	Route each question to the claim, lookup or narrative path; mixed questions run more than one and merge	M	5

FR-5.2	Resolve claim context (payer, product line, DOS, CPTs, ICDs, panel/test) with the PHI filter applied	M	5
FR-5.3	Translate LRN panel and test codes to analytes through the crosswalk	M	5
FR-5.4	Return coverage from the policy version effective on the claim's date of service	M	5
FR-5.5	Where several coverage lines match, return all of them and state which dimension separates them; never pick one	M	5
FR-5.6	Evaluate the claim's ICD codes against the indication group's ICD sets, including ranges and exclusions	M	5
FR-5.7	Check reimbursement rules (quantity, mutual exclusion) against the claim lines, presented as advisory only	P2	5
FR-5.8	Answer narrative questions by retrieval over policy sections, grounded in the criteria tree and quoted	M	5
FR-5.9	Cross-payer comparison for the same code or analyte	P2	5
FR-5.10	Version diff: what changed between two versions of a policy	P2	6
FR-5.11	Every answer cites payer, policy number, version effective date and page, with a link to the source document	M	5
FR-5.12	Where no row exists, answer “this policy does not address that code” — never infer from clinical knowledge	M	5
FR-5.13	As-of-date queries resolve against the version live on that date	M	5
M6 — Change monitoring			
FR-6.1	Scheduled metadata check on registered policy URLs; download only on a real change	P2	6
FR-6.2	Queue changed documents for re-extraction as a new version	P2	6
FR-6.3	Produce a change report: added/removed codes and ICDs, status changes, condition and prior-auth changes	P2	6
FR-6.4	Alert on policies approaching expiry	P2	6

5.2 Non-functional requirements

ID	Requirement	Target and how it is verified
NFR-1	Extraction accuracy	≥95% precision on coverage status, ≥90% recall on codes, measured against the hand-labelled gold set. Gates Phase 2.
NFR-2	Grounding	100% of published rows carry a quote that appears verbatim in the source. Enforced mechanically, not sampled.
NFR-3	Chat latency	Deterministic lookup ≤500 ms; first streamed token ≤1.5 s; full answer p95 ≤6 s.
NFR-4	Pipeline throughput	A 100-page document completes ingest-to-review-queue in ≤15 minutes. A burst of 100 documents drains within 8 hours.
NFR-5	Availability	The chatbot follows the LRN application SLA. The pipeline is asynchronous and may be unavailable for 24 hours with no business impact.
NFR-6	Auditability	Every published row is traceable to model, model version, prompt version, extraction run and named approver. Retained per the LRN records policy.
NFR-7	Security and privacy	PHI boundary enforced at the API; lab scoping on every request; secrets in Key Vault; TLS throughout; RBAC through the LRN role framework.
NFR-8	Cost control	Monthly AI spend under an agreed ceiling with an alert at 80%; per-user daily token budget; per-turn tool-call and row caps.
NFR-9	Re-runnability	Any stage can be re-run from stored intermediates without re-downloading from the payer or re-running OCR.
NFR-10	Maintainability	No payer-specific parsers. Onboarding a new payer that fits an existing archetype is configuration only, completed within one day.
NFR-11	Scalability	Ten times the current corpus without redesign; search tier and worker count are the only scaling levers needed.
NFR-12	Data residency	Documents and model calls stay in the agreed region; no training on customer data; retention configured to the compliance decision.

6. Technology decision — PDF extraction in Python or .NET

> **Decision: the document pipeline is built in Python. The LRN runtime stays .NET.** The boundary is the queue and the database — Python writes candidate rows, .NET owns the review screen, publication, APIs and chat. Neither language crosses that line.

"PDF extraction" is not one job, and the answer differs by sub-task. Splitting it is what makes the decision obvious rather than tribal.

Sub-task	Python option	.NET option	Verdict
Fetch, hash, store	Standard library, Azure SDK	Standard library, Azure SDK	Tie
Text + word coordinates	pdfminer.six, pypdf, PyMuPDF	PdfPig, iText7, commercial suites	Tie — PdfPig is genuinely good
Table geometry	**pdfplumber, camelot** — mature, coordinate-level table reconstruction	No mature equivalent; you hand-roll clustering from word boxes	**Python, decisively**
Cloud OCR / layout	Azure Document Intelligence SDK	Azure Document Intelligence SDK — first-class	Tie
Post-processing (table stitching, segmentation, date scoping, range expansion, master joins)	pandas, regex tooling, rapid iteration	LINQ is capable but verbose; slower to iterate on messy heuristics	**Python**
Structured LLM output	pydantic + provider SDK	System.Text.Json + Azure.AI.OpenAI	Tie
Evaluation harness (gold set scoring, prompt regression, precision/recall reporting)	pandas, scikit-learn, notebooks	No practical equivalent; you build it yourself	**Python, decisively**
Review UI, APIs, chat, auth	Streamlit / FastAPI — a second stack to run and secure	Existing LRN app, roles, audit, deployment	**.NET, decisively**

Weighted decision matrix

Criterion	Weight	Python	.NET	Note
Table and layout extraction maturity	25%	**5**	2	The determination matrix archetype depends on this
Post-processing and data wrangling	15%	**5**	3	Stitching, scoping, merging are heuristic and change often
Evaluation and prompt regression tooling	15%	**5**	2	The gold set gates every model change
Team skill and existing footprint	15%	4	**5**	Team is .NET-first, but already runs Python payer-policy workers
Integration with the LRN runtime	10%	2	**5**	Only matters for the runtime half, which stays .NET
Deployment and operations	10%	3	**4**	Both containerise on the existing VM
Licensing cost and risk	10%	4	3	See the licence note below — both stacks have an AGPL trap
Weighted score	**100%**	**4.25**	**3.20**	

> **Licensing trap in both stacks — check before writing code. PyMuPDF** is AGPL-3.0; its network clause can reach a server application, so use it only under a purchased Artifex licence. The MIT-licensed **pdfplumber** and **pdfminer.six** carry no such obligation and are the default here. On the .NET side, **iText7** is AGPL or paid commercial; **PdfPig** is Apache-2.0 and safe. Recommended set: **pdfplumber + pdfminer.six (MIT)**, with Document Intelligence for anything they cannot handle.

> **Cost is not the deciding factor, and it is honest to say so.** Running Document Intelligence Layout on *every* page of the expected corpus costs roughly $10 per 1,000 pages — on the order of $120–250 a year at the volumes in section 17. A .NET-only build that leaned entirely on Document Intelligence and skipped local table parsing would therefore be affordable. It would be rejected on **accuracy verification and iteration speed**, not on price: without pdfplumber there is no cheap second opinion on a matrix page, and without the Python evaluation stack, measuring a prompt change against the gold set becomes a project of its own.

What would change this decision

- **If no Python engineer is available for the build.** Then a .NET pipeline calling Document Intelligence for all pages is viable — budget roughly 25–30% more effort for table reconstruction and the evaluation harness, and accept slower prompt iteration.
- **If the corpus turns out to be entirely narrative** with no determination matrices, the table-geometry advantage disappears and the two stacks converge. The sample says otherwise: two of seven payers use matrices.
- **Not a reason to switch:** the team being "a .NET shop". The pipeline is a batch process behind a queue with no UI, no user session and no shared runtime; it is the cheapest possible place in the architecture to run a second language.

7. Technology stack decisions

Layer	Chosen	Alternatives considered	Why this one
Document pipeline runtime	**Python 3.12**, containerised workers	.NET 8 workers; Azure Functions; Logic Apps	Table geometry and evaluation tooling; existing Python worker footprint. Functions rejected — long-running 348-page documents fit a worker better than a function timeout.
PDF text and tables	**pdfplumber + pdfminer.six** (MIT)	PyMuPDF (AGPL), PdfPig, iText7, Aspose	Permissive licence, mature table reconstruction, coordinates for highlighting.
OCR and complex layout	**Azure AI Document Intelligence**, Layout model, selective	Tesseract; DI on every page; vision-LLM on page images	Only matrix and scanned pages are escalated. Vision-LLM rejected: cost and accuracy both worse on 348-page documents.
Extraction model	**Long-context frontier model**, provider decided by benchmark in Phase 2	Azure OpenAI GPT family; Claude via API; open-weight self-hosted	Precision on coverage status outweighs token savings. Kept behind an `IPolicyExtractor` interface so the provider is swappable.
Chat and classification model	**Smaller, faster model** on the same provider	One model for everything	Section classification, code normalisation and chat wording do not need frontier capability; the cost and latency difference is large.
Retrieval	**Azure AI Search**, hybrid	SQL full-text; pgvector; in-memory vectors	Lexical matching for CPT and ICD codes plus semantic matching for narrative questions, in one index with metadata filters.
Structured store	**SQL Server** (existing), Azure SQL MI later	Azure SQL MI now; PostgreSQL	The coverage database is a normal relational workload. Migration is a separate infrastructure decision, not a prerequisite.
Document store	**Azure Blob Storage**	File share on the VM; SQL FILESTREAM	Immutable originals, lifecycle rules, SAS links for the review screen.
Queue	**Azure Service Bus**	Storage Queues; RabbitMQ; SQL table as a queue	Sessions, dead-lettering and retry semantics out of the box.
Application and APIs	**ASP.NET Core** in the existing LRN app	FastAPI; Streamlit for review	Reuses authentication, the role and menu framework, audit and deployment.
Chat UI	**Razor partial + vanilla JS, SSE streaming**	Angular/React SPA panel	Matches the existing LRN front end and the CPT & Panel Lookup agent pattern.
Secrets and identity	**Azure Key Vault** + existing LRN roles	Entra ID for all identity	Key Vault is already in use. Entra ID remains an open decision (D-3).
Observability	**Application Insights** + audit tables	Serilog to file; third-party APM	Traces the pipeline and the chat turn, and the audit tables answer the business questions.
Hosting	**Docker on the existing Windows Server 2022 VM**	Azure Container Apps; AKS; App Service	The containerisation work is already in flight; Azure hosting is the later step, not the first one.

8. Solution architecture and deployment

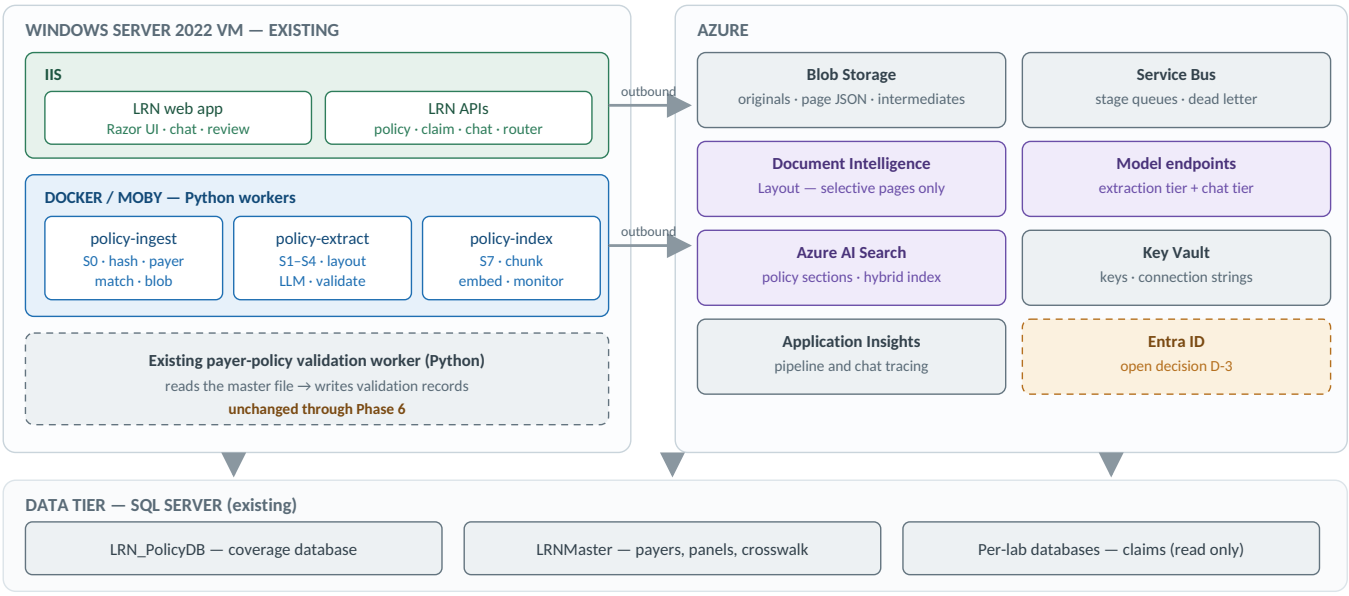

Figure 3 — Deployment view. Nothing new is hosted outside the existing VM and the Azure subscription.

- **No new servers.** The three Python workers are containers on the VM that already runs the LRN applications, alongside the existing validation worker. Scaling is container replicas, not machines.
- **All Azure traffic is outbound.** No inbound path from Azure into the VM is required, which keeps the network review short.
- **One database for coverage.** `LRN_PolicyDB` is a new database on the existing SQL Server instance; LRNMaster keeps the payer, panel and crosswalk data it already owns; per-lab claim data stays read-only to this feature.
- **Environments.** Dev and test workers run against a separate Blob container, Service Bus namespace and database, using the free Document Intelligence tier where page limits allow, and the smaller model tier for both roles.
- **CI/CD.** Worker images build and publish through the existing pipeline; the LRN app deploys as it does today. Prompt and schema versions are release artefacts, tagged and recorded in the audit table.

9. Ownership — AI, Python, .NET, human

HUMAN .NET AI SERVICE PYTHON DATA This colour coding is used in every diagram in this document.

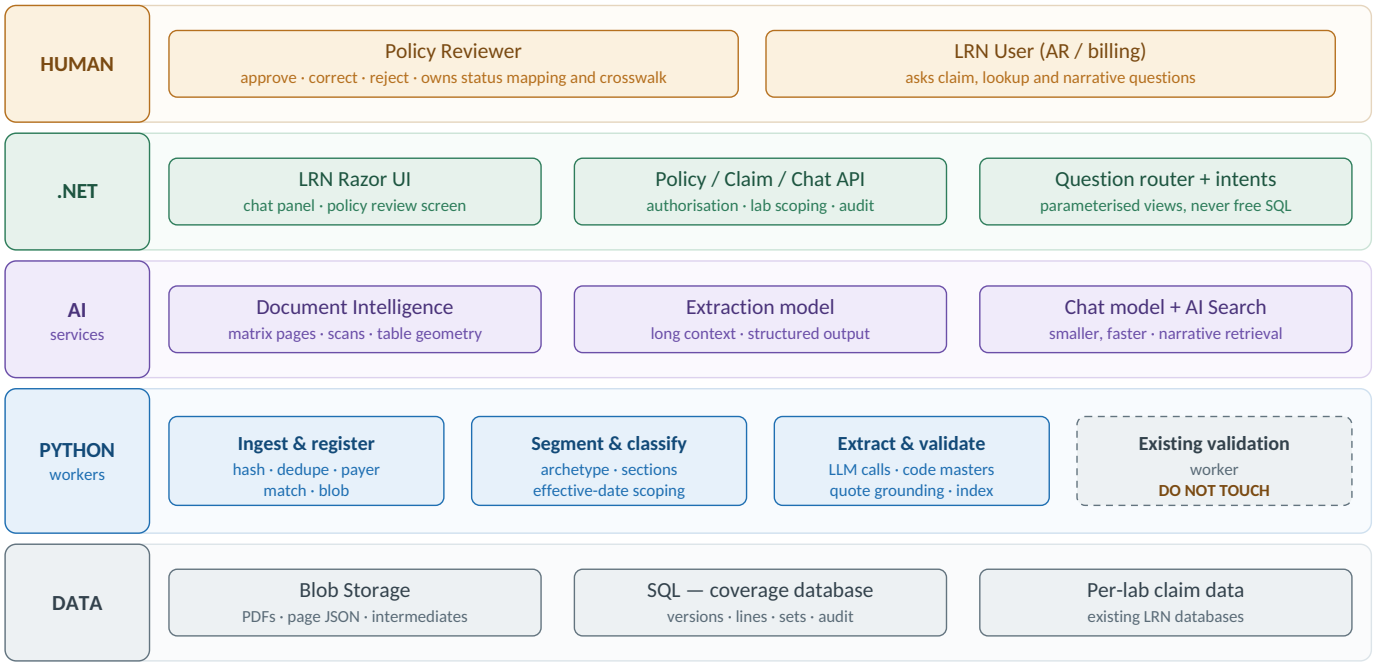

Figure 4 — Component map. Each band is one responsibility owner.

Layer	Owns	Never does
AI	Reads the segmented sections, produces canonical facts with page and quote receipts, explains policy in natural language	Decide coverage, invent a code, write to SQL, run SQL, approve anything
Python	Ingestion, segmentation, archetype detection, date scoping, LLM orchestration, validators, deterministic merge, indexing	Business rules, user authorisation
.NET	APIs, claim access, lab scoping, question routing, parameterised lookup intents, chat streaming, review screen	Generate SQL from natural language; call the model outside the tool contract
Human	Approves every published row; owns the status synonym table and the LRN panel/test-to-analyte crosswalk	—
SQL	Authoritative coverage facts, versions, provenance and audit	—
AI Search	Narrative retrieval over policy sections and source evidence	Be treated as the source of truth for coverage status

10. The grain — the single most important decision

The whole design turns on what one row of the coverage table means. Get the grain right and the ICD explosion, the pathogen dimension and the contract-tier problem all resolve themselves.

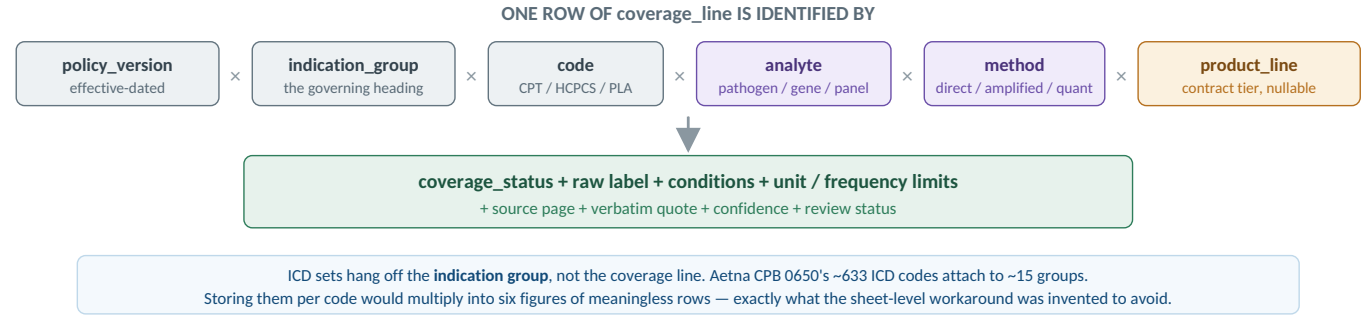

Figure 5 — The grain of a coverage row. Analyte and method are what v2.0 was missing.

Canonical coverage status enum

`COVERED` `COVERED_WITH_CRITERIA` `NOT_COVERED` `NOT_MEDICALLY_NECESSARY` `INVESTIGATIONAL` `NOT_REIMBURSABLE` `PRIOR_AUTH_REQUIRED` `NO_STATEMENT`

> **`NO_STATEMENT` is deliberate and load-bearing.** "This policy is silent on that code" is a different answer from "this policy does not cover that code", and both the database and the chatbot must be able to say which one applies. An unmapped raw label never becomes a guess at the enum — it becomes a reviewer exception.

11. Data model

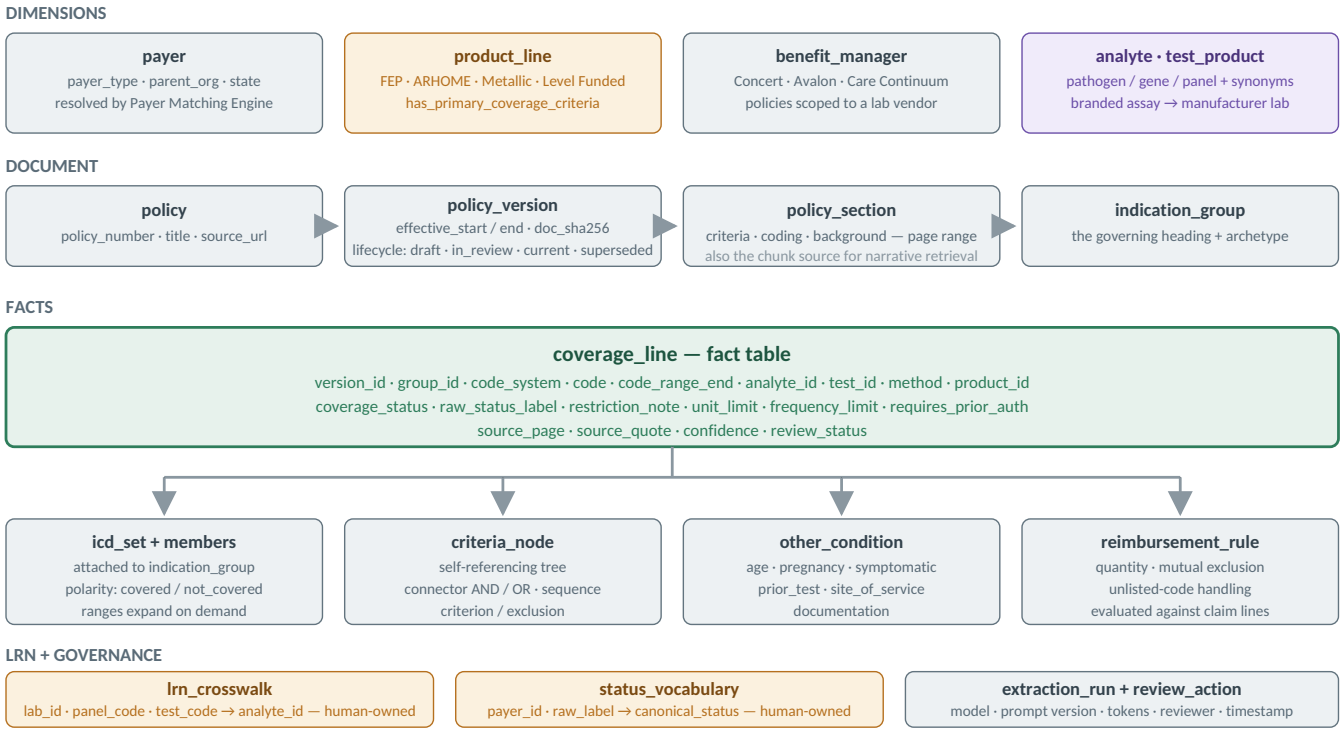

Figure 6 — Data model. Two tables are human-owned and never written by the AI.

Two tables the AI never writes. `status_vocabulary` maps each payer's raw wording to the canonical enum; `lrn_crosswalk` maps LRN panel and test codes to analytes so a claim can reach a coverage line. Both are maintained by billing operations through the LRN admin screens, and both are the reason the system can be audited.

12. Pipeline

Eight stages, each independently re-runnable against stored intermediates. If the extraction prompt improves in month six, stages 0–2 do not re-run and nothing is re-downloaded from a payer site.

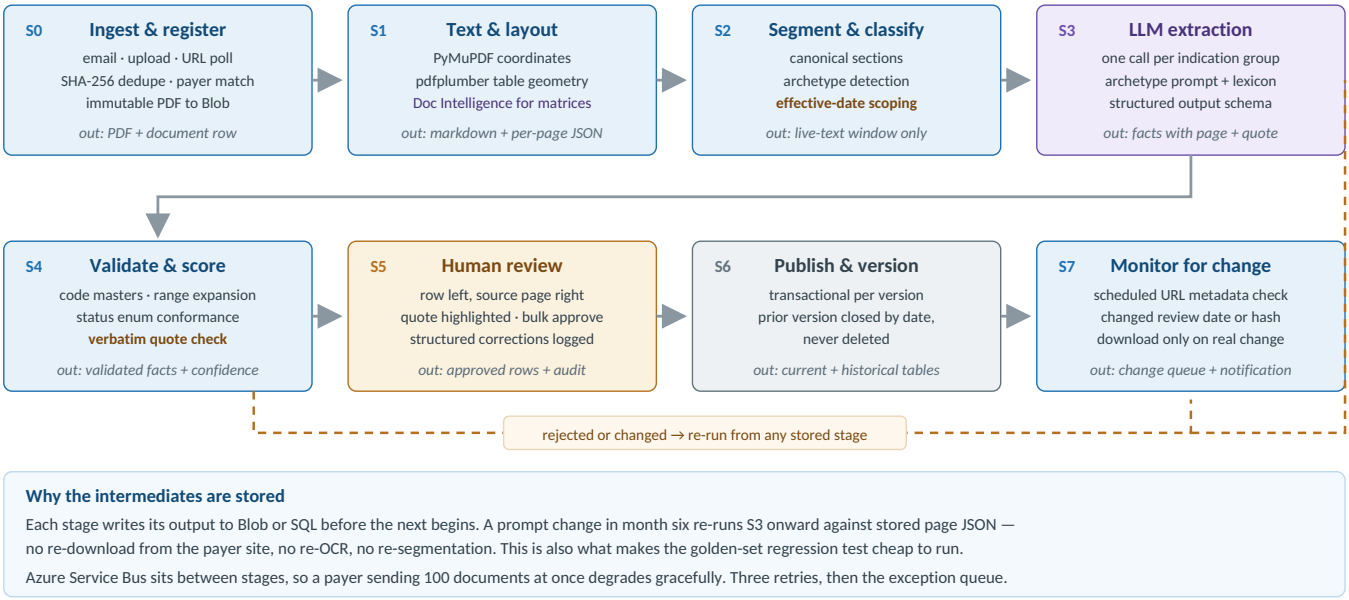

Figure 7 — Eight-stage pipeline. Stage colour shows which layer owns it.

Extraction contract

The prompt is not the interesting part — the contract around it is. The model receives one indication group at a time, is told which archetype it is reading, is handed that payer's status lexicon, and is forbidden from emitting any fact it cannot quote.

- **Never infer a code.** If a CPT or ICD code does not appear literally in the supplied text, it does not exist. No expansion from clinical knowledge.
- **Every fact carries a receipt.** Each row must include `source_page` and a `source_quote` that appears verbatim in the input. A validator checks this mechanically; unquotable rows are discarded, not trusted.
- **Status comes from the nearest governing heading**, and the raw label is reported alongside the canonical value so the mapping stays auditable.
- **Ambiguity is an output, not a guess.** Where the document does not clearly state a position, emit `NO_STATEMENT` with a flag and let the reviewer decide.
- **Preserve, don't paraphrase.** Criteria trees keep their connectors; code ranges keep their range form; bracketed qualifiers stay in their own field.

Output is forced through a JSON schema via structured output rather than requested in prose, so a malformed response is a retryable error rather than a parsing problem. The response object mirrors the data model exactly, which means the loader is a straight insert with no interpretive layer between the model and the database.

Model selection

Extraction and chat are different jobs and should not be forced onto one model. **Extraction** runs on a long-context frontier model — the Kansas and Aetna documents need a whole coding section in one window, and precision on coverage status is worth more than the token saving. **Interactive chat and the cheap deterministic steps** (section classification, code normalisation) run on a smaller, faster model. Both providers are benchmarked against the gold set in Phase 2, and measured accuracy on these documents picks the extraction model rather than the decision being made now.

13. Chatbot — three question types

Three kinds of question arrive, and answering all three with one mechanism is how these systems go wrong. A router classifies the question and picks the path; mixed questions run more than one path and merge.

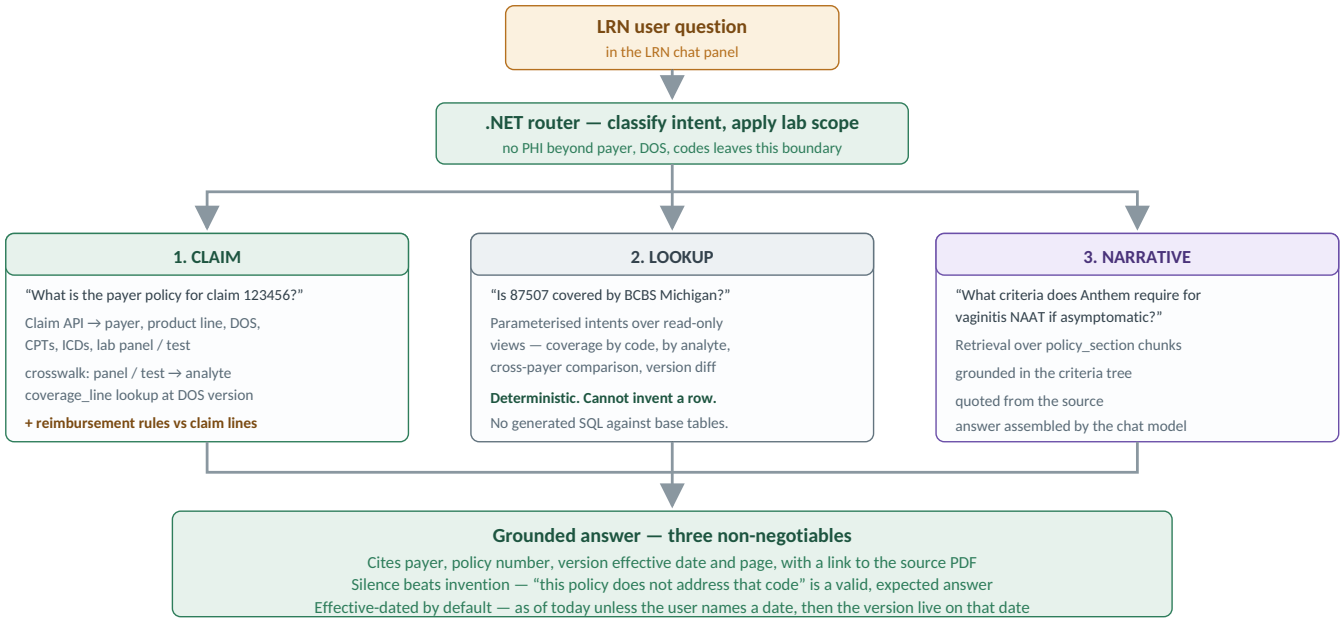

Figure 8 — Question router. Lookup is deterministic; only narrative wording is generated.

> **New consequence of the finer grain.** A claim gives us payer, product line, date of service, CPTs and ICDs — but it does **not** reliably give us the analyte or the method. So a lookup by CPT alone can legitimately return several coverage lines at different statuses, as with BCBS Michigan 87797. The bot must not pick one. It returns every matching line, states which dimension separates them, and asks for the missing detail — for example, "covered for these organisms by amplified probe; investigational for quantification — which was performed?" The LRN panel/test crosswalk is what narrows this automatically where the lab's own test definition already implies the analyte.

Claim answers also check the reimbursement rules — quantity limits and mutual-exclusion pairs such as the BCBS Illinois "one, but not both" influenza rule — against the actual claim lines. This is presented as advisory information with its source, never as an adjudication decision.

14. Review and promotion

The reviewer works a queue scoped to one policy version. Each screen shows the extracted row and the rendered source page side by side, with the quoted sentence highlighted, so verification is a glance rather than a hunt. Rows above the confidence threshold can be bulk-approved; anything the validator flagged surfaces first.

Edits are captured as **structured corrections, not free text**, which makes the correction log double as a prompt-improvement dataset — the rows reviewers change most often tell you exactly which prompt rule is weak.

Nothing reaches the current tables without approval. The load is transactional per policy version: a version is either wholly published or wholly held, so the database is never half-updated mid-review. The previous version is closed by effective date rather than deleted, so "what did this payer cover last March" stays answerable.

> **Where this document overrides the source review.** The source review proposed a standalone Streamlit or FastAPI review app. This design keeps the review screen **inside LRN**: the reviewers are already LRN users, the role and audit framework exists, and the crosswalk maintenance belongs beside the panel data. If Phase 3 is at risk, a temporary Streamlit harness on the same VM is an acceptable fallback for internal reviewers only — not a parallel permanent UI.

> **Review capacity is the real constraint, not AI cost.** Aetna CPB 0650 alone will generate on the order of a few hundred coverage lines. Phase 3 must measure minutes-per-document before the payer list is widened, and the decision to allow any rule type to skip full review is a Phase 6 decision taken against measured accuracy — never an assumption.

15. Security and compliance

- **PHI boundary.** The models receive payer, product line, date of service, CPT, ICD, analyte and claim reference only — never patient name, MRN, DOB or member ID. This requires a BAA and no-training / no-retention configuration on the endpoints, and it is a Phase 0 blocker.
- **Lab scoping.** Coverage data is payer-level and shared; claims, panels and users are lab-scoped. Every chat and claim request is filtered by the user's lab entitlement.
- **Roles.** New permission keys in the existing LRN role and menu framework: `PolicyChat`, `PolicyReview`, `PolicyAdmin`. Whether to move to Entra ID is a separate open decision.
- **Audit.** `extraction_run` and `review_action` make every published number traceable to a model version, a prompt version and a named approver. Chat is audited separately with per-turn tool calls and token cost.
- **Ceilings.** Tool-call limit per turn, row caps per result, wall-clock limit, per-user daily token budget, and a feature flag that renders existing screens unchanged when off.

16. Performance and efficiency design

16.1 Where the time goes

Stage	Dominant cost	Budget	Lever
S1 layout extraction	Document Intelligence round trip on escalated pages	≤3 s per escalated page	Escalate selectively; submit page ranges, not whole documents; run pages concurrently
S2 segmentation	Local CPU only	≤30 s per document	Pure Python, no service call — this is why it is cheap to re-run
S3 extraction	Model latency and tokens	≤60 s per indication group	Segmentation removes 60–80% of the text before the call; groups run in parallel
S4 validation	Code master lookups	≤5 s per document	Load masters into memory once per worker, not per row
Chat — lookup path	SQL over indexed views	≤500 ms	Parameterised intents, covering indexes, row caps
Chat — narrative path	Retrieval plus model wording	first token ≤1.5 s, p95 ≤6 s	Filter the index by payer and version before ranking; stream the answer

16.2 The efficiency lever that matters most

> **Segmentation is the single biggest efficiency decision in the design.** In Aetna CPB 0650 the criteria and coding sections are roughly 17% of the extracted text. Sending only those sections cuts the extraction token bill by around 80% on that document, shortens every call, and — the important part — measurably improves accuracy, because the model is not asked to find three coverage statements inside 290 pages of literature review. Cost and quality move in the same direction here, which is rare enough to be worth stating out loud.

16.3 Throughput and concurrency

- **Worker concurrency** is bounded by the Document Intelligence transaction limit and the model endpoint's tokens-per-minute quota, not by the VM. Set worker replica counts from those two numbers and let Service Bus absorb the rest.
- **Back-pressure over failure.** A burst of 100 documents queues; it does not fail. Each stage retries three times with exponential backoff before dead-lettering to the exception queue with a notification.
- **Idempotency.** Every stage is keyed on document and version, so a redelivered message is safe. This is what allows aggressive retries.
- **Re-runnability is a performance feature.** A prompt change in month six re-runs S3 onward from stored page JSON — no re-download, no re-OCR, no re-segmentation. It also makes the nightly gold-set regression cheap enough to actually run.

16.4 Caching

- **Prompt caching** on the static prefix — system rules, canonical schema, archetype instructions, payer status lexicon — which is the majority of every extraction call.
- **Answer caching** for deterministic lookups keyed on payer, product line, code and policy version; invalidated when a version is published.
- **Code masters and the status vocabulary** held in worker memory with a version stamp.

17. Cost model

> Rates below are indicative list prices at the time of writing and must be re-checked against the Azure pricing pages for your region and agreement before any budget is approved. Model rates in particular move several times a year.

17.1 Rate card

Item	Indicative rate	Notes
Document Intelligence — Read (OCR)	~$1.50 per 1,000 pages	Text only, no structure
Document Intelligence — Layout	~$10 per 1,000 pages	Tables, sections, bounding boxes. Batch is billed at the same rate — there is no batch discount
Document Intelligence — free tier	500 pages/month, first 2 pages per request	Useful for dev and test only
Extraction model	~$2–5 per 1M input, ~$10–20 per 1M output	Frontier tier; confirm at benchmark time
Chat / classification model	~$0.10–0.40 per 1M input	Small tier; cached input is cheaper again
Azure AI Search	~$75/month Basic, ~$250/month Standard S1	Basic is sufficient for this corpus size
Service Bus (Standard)	~$10/month	Well inside the base throughput allowance
Blob Storage	a few dollars per month	Originals plus intermediates are small
Compute	$0 marginal	Containers on the existing VM
SQL Server	$0 marginal	New database on the existing instance

17.2 Worked example

Assumptions, to be replaced with your real numbers: an initial load of **200 policy documents averaging 60 pages** (12,000 pages), of which about 25% of pages are escalated to Layout; a steady state of **40 documents a month** (2,400 pages); roughly 600 tokens of text per page, reduced to about 20% by segmentation; a two-pass extraction with retries, so input tokens are counted at roughly twice the segmented text.

Line	Year 1 initial load	Steady state / month	Basis
Document Intelligence — Read on all pages	~$18	~$4	12,000 and 2,400 pages
Document Intelligence — Layout on ~25% of pages	~$30	~$6	3,000 and 600 pages
Extraction model tokens	~$35–70	~$7–14	~3M input, ~1M output for the initial load
Chat and classification tokens	~$10	~$5–20	Scales with chat usage, not corpus size
Azure AI Search (Basic)	—	~$75	Fixed tier cost
Service Bus, Blob, Key Vault	—	~$15	Fixed
Total AI and platform	**~$95–130 one-off**	**~$110–135 / month**	
Human review labour	**~150 hours**	**~30 hours / month**	200 documents at ~45 minutes each

> **The headline finding: AI spend is not the cost of this project.** Platform and model costs land around a hundred dollars a month at this volume — less than the search tier, and a rounding error against engineering time. The two real costs are **build effort** (section 18) and **review labour**. Every design decision that reduces review minutes per document is worth more than every decision that reduces tokens. This is also why bulk approval stays disabled until Phase 6: buying review capacity with unverified accuracy is the one trade in this project that could actually cost money.

17.3 Cost controls

- Monthly budget alert at 80% of the agreed ceiling, split by extraction and chat.
- Per-user daily token budget and per-turn tool-call ceiling on the chatbot.
- Selective Layout escalation — whole-document Layout is the easy mistake and costs roughly seven times more per page than Read.
- Dev and test on the free Document Intelligence tier and the small model tier.
- Token cost recorded per extraction run, so cost per published row is a reportable number rather than a guess.

18. Development breakdown

18.1 Work packages

Effort is in developer-days for one competent engineer in that stack, excluding review meetings and the business time in the Review track.

WP	Work package	Stack	Days	Phase	Requirements
WP1	Azure provisioning, Key Vault wiring, container skeleton, CI/CD for workers	DevOps	5	0	NFR-7, NFR-12
WP2	Schema DDL, code masters load, status vocabulary seed, audit tables	SQL	8	0	NFR-6
WP3	Ingestion worker: upload, hashing, three-key dedupe, payer matching, Blob, queue	Python	8	1	FR-1.1, 1.4–1.7
WP4	Layout extraction: text with coordinates, table geometry, selective Document Intelligence escalation	Python	10	1	FR-2.1–2.3
WP5	Segmentation, archetype detection, effective-date scoping, normalised output	Python	12	1	FR-2.4–2.7
WP6	Extraction schemas and archetype prompts; provider abstraction	Python	15	2	FR-3.1–3.3
WP7	Validators: code masters, range expansion, enum mapping, verbatim grounding, trust score, retry	Python	10	2	FR-3.4–3.7, 3.9, 3.10
WP8	Deterministic merge of matrix, criteria block and code appendix into coverage lines	Python	8	2	FR-3.8
WP9	Evaluation harness: gold set loader, precision/recall scoring, regression report	Python	8	2	NFR-1, NFR-2
WP10	Review screen: queue, side-by-side PDF render, quote highlight, approve/edit/reject	.NET	18	3	FR-4.1–4.4
WP11	Transactional publish, versioning, supersession, correction capture, audit writes	.NET	8	3	FR-4.6, 4.7, 4.9
WP12	Admin screens: status vocabulary and panel/test-to-analyte crosswalk	.NET	6	3	FR-4.8
WP13	Excel workbook mapping, re-extraction and variance report	Python	10	4	—
WP14	Chunking, embeddings, AI Search index with metadata filters, hybrid query	Python	8	5	FR-5.8
WP15	Parameterised lookup intents over read-only views	.NET	10	5	FR-5.4, 5.9, 5.13
WP16	Claim path: context API with PHI filter, crosswalk resolution, multi-line disambiguation, ICD set evaluation	.NET	12	5	FR-5.2, 5.3, 5.5–5.7
WP17	Question router, grounding rules, citation assembly, budgets and ceilings	.NET	10	5	FR-5.1, 5.11, 5.12, NFR-8
WP18	Chat panel in LRN with SSE streaming, chat audit tables	.NET	10	5	FR-5.1, NFR-6
WP19	Change monitoring: URL polling, change queue, version diff report, expiry alerts	Python	10	6	FR-6.1–6.4
WP20	Email ingestion	Python	5	6	FR-1.2
WP21	Observability, cost telemetry, runbooks, exception-queue tooling, hardening	Both	8	all	NFR-3–NFR-5, NFR-8
Total			**199**	**~40 developer-weeks**	

18.2 Split by stack

Stack	Days	Share	Work
Python	104	52%	Ingestion, layout, segmentation, extraction, validators, merge, evaluation, index, change monitoring
.NET	74	37%	Review screen, publication, admin screens, lookup intents, claim path, router, chat panel
SQL / DevOps	13	7%	Schema, masters, provisioning, CI/CD
Shared	8	4%	Observability and hardening

18.3 Team and calendar

Role	Allocation	Responsibility
Python engineer	1.0 FTE	WP3–WP9, WP13, WP14, WP19, WP20 — the document pipeline
.NET engineer	1.0 FTE	WP10–WP12, WP15–WP18 — review screen, APIs, chatbot
SQL / DevOps	0.3 FTE	WP1, WP2, deployment, backup and retention
Billing SME / reviewer	0.5 FTE	Gold set, status mapping, crosswalk, review of every phase gate
Tech lead	0.2 FTE	Model benchmark, architecture decisions, gate sign-off

At two full-time engineers the 199 days land at roughly **20 calendar weeks**, which matches the phase durations in section 19. Two dependencies set the critical path and cannot be parallelised away: **WP5 must complete before WP6 is meaningful** (there is no point tuning prompts against unsegmented documents), and **WP10 must complete before any policy is published**, so the Phase 5 chatbot has nothing to answer from until the review screen exists. The Python and .NET tracks are otherwise independent after Phase 1, which is what allows two engineers to work without blocking each other.

> **The one estimate most likely to be wrong is WP6.** Prompt and schema work against four archetypes is exploratory, not mechanical — 15 days assumes the gold set exists on day one and the segmentation from WP5 is clean. If Phase 2 misses its accuracy gate, the overrun lands here, and the correct response is to narrow the archetype scope rather than to lower the gate.

19. Phased delivery

Each phase ends at a gate you can observe, not a status meeting. Durations assume part-time effort alongside existing platform work. Every phase carries a **Review** track (decisions and sign-off) and a **Development** track.

Figure 9 — Roadmap. Phase 7 (master-file assist) sits beyond this line and needs a separate decision.

Phase 0 — Foundations

1–2 weeks

Review track	Development track
Compliance sign-off: BAA, no-retention, PHI redaction rule, data residency — **blocker**	Provision Azure SQL, Blob containers and Key Vault; wire to the existing VM
Agree the canonical status enum and confirm the mapping for all 13 observed labels	Schema v1 DDL including audit and provenance tables
Confirm identity approach (existing LRN roles vs Entra ID)	Load CPT / HCPCS / PLA and ICD-10 code masters
Confirm the four archetypes cover the payers you actually receive	Seed the seven-payer status synonym table from the sample documents

Gate: sample coverage rows can be inserted, versioned and queried end to end by hand.

Phase 1 — Ingest & segment

2–3 weeks

Review track	Development track
Agree exception-queue ownership and escalation	Blob ingestion with hashing, dedupe and document registration
Confirm which payer mailboxes and policy URLs are in scope	Layout extraction (PyMuPDF + pdfplumber) with the Document Intelligence fallback
	Section classifier and archetype detector
	Effective-date scoping for documents carrying historical blocks
	Payer Matching Engine integration

Gate: all 14 sample documents segment correctly, and the Arkansas Lyme policy resolves to its live block only.

Phase 2 — Extraction engine

3–4 weeks

Review track	Development track
Billing team hand-labels a gold set — one document per archetype	JSON schemas and archetype-specific prompts
Weekly accuracy readout against the gold set	Validators: code masters, range expansion, enum conformance, verbatim grounding
Sign off the provider and model choice on measured results, not vendor claims	Deterministic merge: matrix + criteria + appendix → coverage lines
	Provider and model benchmark against the gold set

Gate: ≥95% precision on coverage status and ≥90% recall on codes, measured against the gold set.

Phase 3 — Review UI and publish

2–3 weeks

Review track	Development track
Reviewer team confirms the workflow and turnaround SLA	Review queue in LRN with side-by-side source rendering and quote highlighting
Measure minutes-per-document on real policies	Approve / edit / reject with structured correction capture
Billing ops begin the panel/test-to-analyte crosswalk	Transactional promotion to current, with effective-dated supersession
	Status vocabulary and crosswalk maintenance screens

Gate: a member of the policy team takes one new policy from PDF to published rows without assistance.

Phase 4 — Excel migration and reconciliation

2 weeks

Review track	Development track

Business reviews the variance report and rules on each disagreement	Map the existing workbook's payer sheets, ICD blocks and other-conditions into the schema
Decide which source wins where the sheet and the document disagree	Re-extract the policies the workbook already covers and reconcile
	Variance report: where the sheet and the source document disagree

Gate: the workbook's content exists in the database, and every variance is either explained or corrected.

Phase 5 — Chatbot in LRN 4–5 weeks

Review track	Development track
Build a held-out set of ~50 questions with known answers, including deliberate "not addressed" cases	Parameterised lookup intents over read-only views
Reviewers score answers for correctness and citation accuracy	Narrative retrieval over policy sections with metadata filters and hybrid search
Sign off the grounding rules and the refusal behaviour	Claim path: claim context API with PHI filtering, crosswalk resolution, multi-line disambiguation
Decide the pilot user group	Question router, chat panel in LRN, chat audit tables and token budgets

Gate: the held-out set passes with ≥95% citation accuracy, zero fabricated codes, and correct "not addressed" behaviour on every silent case.

Phase 6 — Change monitoring and scale 3–4 weeks

Review track	Development track
Business validates the version-diff report against a real policy update	Scheduled URL metadata polling and change queue
Decide which rule types may skip full review, based on Phases 2–3 measured accuracy	Version diffing: added/removed codes and ICDs, status changes, condition changes
	Automated email ingestion, expiry alerts, notifications
	Onboard the remaining payers

Gate: a real payer update is detected, diffed, reviewed and published without anyone noticing it by hand first.

Phase 7 — Master-file assist future · separate decision

AI **proposes** master-file updates from approved coverage data; a human approves; the existing Python validation worker consumes the file exactly as it does today. The AI never writes to the production master file.

20. Metrics

Area	Metric	Target
Extraction	Coverage-status precision; code recall; ICD-set recall; verbatim-grounding pass rate	≥95% / ≥90% / ≥90% / 100% (ungrounded rows are discarded)
Review	Minutes per document; share of rows approved with no change; most-corrected prompt rules	Trending down / trending up / reviewed monthly
Chatbot	Answer accuracy, citation accuracy, fabricated-code count, correct “not addressed” rate	≥90% / ≥95% / zero / 100%
Business	Hours spent reading payer documents and maintaining the workbook; share of policy questions answered without a human	Baseline vs. post-launch

21. Decisions taken, and what is still open

21.1 Decided in this document

#	Decision	Rationale
D-1	**PDF extraction is built in Python**, not .NET	Table geometry and evaluation tooling; weighted score 4.25 vs 3.20 (section 6). Cost is not the differentiator.
D-2	**The LRN runtime stays .NET** — review screen, APIs, router and chat panel	Reuses authentication, roles, audit and deployment. The Streamlit option in the source review is rejected as a permanent UI.
D-3	**pdfplumber + pdfminer.six (MIT)** as the PDF libraries	Avoids the PyMuPDF AGPL network clause without buying a commercial licence.
D-4	**Document Intelligence used selectively**, not on every page	Layout costs roughly seven times Read per page; escalate matrix and scanned pages only.
D-5	**Two model tiers** — frontier for extraction, small for chat and classification	Precision where it matters, cost and latency where it does not.
D-6	**SQL Server now**, Azure SQL MI as a later infrastructure decision	A new database on the existing instance removes a dependency from the critical path.
D-7	**Containers on the existing VM**, not new hosting	No new servers, outbound-only Azure traffic, aligns with the containerisation already in flight.
D-8	**Coverage grain** = version × indication group × code × analyte × method × product line	Derived from the fourteen sample documents; see section 10.
D-9	**Bulk approval stays disabled until Phase 6**	Skipping review is only safe against measured accuracy, never against an assumption.

21.2 Still open

#	Decision	Owner	Needed by
O-1	Compliance sign-off: BAA, no-training / no-retention, PHI redaction rule, data residency	Compliance	Phase 0 — blocker
O-2	Extraction model provider — decided by the gold-set benchmark, not chosen up front	Tech lead	Phase 2
O-3	Identity: existing LRN role framework or Microsoft Entra ID	Architecture	Phase 0
O-4	Which payers beyond the seven sampled are in the first production wave	Billing operations	Phase 1
O-5	Who owns policy review, and the daily review capacity	Billing operations	Phase 3
O-6	Who maintains the status vocabulary and the panel/test-to-analyte crosswalk	Billing operations	Phase 3
O-7	Where the workbook and the source document disagree, which one wins	Billing operations	Phase 4
O-8	Monthly AI budget ceiling and per-user token budget	Management	Phase 2
O-9	Whether a Python engineer is available for the full build — if not, re-open D-1 with a 25–30% effort premium	Management	Phase 0

22. Summary

> **Document → Python ingest → Blob → layout extraction → segment by archetype and effective date → LLM extraction with quote receipts → validation → human approval → versioned coverage database + policy sections in AI Search → question router → .NET API → LRN chatbot**, with the existing master file and Python validation worker untouched until Phase 7.

Two rules carry the whole design. **The LLM is the intelligence layer, not the coverage database** — SQL holds the facts, retrieval holds the evidence, Python does the processing, .NET enforces security and business rules, and the human reviewer remains the approval authority. And **the grain is everything**: a coverage row is a payer's position on one code, for one analyte, by one method, under one contract tier, in one policy version. Every hard problem in the fourteen sample documents — the 633-ICD explosion, the pathogen dimension, the contract-tier split, the same code at two statuses — is a consequence of getting that one decision right or wrong.